Day - 14: PWM

▼ What is PWM | Pulse Width Modulation | Double Edge PWM

1. Introduction & Objective

- **The Goal:** To generate a hardware interrupt using the Pulse Width Modulation (PWM) peripheral.
- **Core Logic:** Generating an interrupt from PWM is very similar to generating an interrupt from a standard timer. A PWM signal consists of an ON period, an OFF period, and a total time period.

2. Match Registers and Timer Counter

To understand when the interrupt triggers, developers must understand how PWM utilizes match registers:

- **MR0 (Match Register 0):** This register holds the **total time duration** of the PWM cycle ($T_{Total} = T_{ON} + T_{OFF}$).
- **Other Match Registers (e.g., MR1 to MR6):** These registers hold the duty cycle, which dictates the specific **ON time** period.
- **The Trigger:** The PWM peripheral uses a main Timer Counter (TC). Whenever the active Timer Counter reaches the exact value stored in **MR0**, the system completes one full PWM cycle and can generate an interrupt.

3. Essential PWM Interrupt Registers

To configure this behavior, two specific registers must be managed:

- **PWM MCR (Match Control Register):** Controls how the match registers behave.
    - **Bit 0:** Controls the PWM MR0 Interrupt Enable. To generate the interrupt, this bit must be set so that an interrupt triggers when the TC matches MR0.
- **PWM IR (Interrupt Register):** Acts as the status flag.

- **Bit 0:** Whenever an interrupt successfully occurs, this bit will be set.
- **Crucial Rule:** The programmer must explicitly clear this bit inside the Interrupt Service Routine (ISR) to reset the system for the next interrupt.

4. Programming Implementation (3 Key Additions)

Writing the code for a PWM interrupt is nearly identical to standard PWM configuration, but requires three essential additions:

1. **Configure MCR (Interrupt & Reset Mode):** Unlike standard PWM which only enables reset mode, you must enable **both** interrupt mode and reset mode on MR0. This ensures the timer resets and an interrupt fires simultaneously at the end of every cycle.
2. **Enable the NVIC:** You must instruct the processor's **Nested Vectored Interrupt Controller (NVIC)** to enable the PWM interrupt line and assign it an interrupt priority level.
3. **Create the PWM Handler (ISR):** When the interrupt occurs, the code jumps to this handler. In this exercise, the handler prints a message ("PWM IRQ") to the terminal and then immediately **clears the PWM interrupt flag** so the program can continue.

5. Execution Flow & Debugging

- **Main Setup:** In the main code, MR0 is set to exactly **1 second** to make the visual output easy to observe.
- **The Loop:** The main `while(1)` loop is configured to continuously generate a PWM signal that cycles through different duty cycles (25%, 50%, 75%, and 95%).
- **Verification:** Using a software debug window, the programmer can simultaneously watch the active PWM signal and the UART terminal. Every time the PWM completes one full 1-second cycle, the terminal instantly prints the interrupt message, proving the ISR successfully executed exactly when TC matched MR0.